

TP n°6

Programmation orientée objet en Java

Surcharge, polymorphisme statique

Temps de réalisation: 1h30

Vous travaillez dans une équipe développant une bibliothèque interne de calcul utilisée dans plusieurs projets : simulation numérique, optimisation, et analyse de données. Cette bibliothèque permet de manipuler des **expressions mathématiques sous forme d'arbres** (AST - Abstract Syntax Tree). Chaque expression est représentée par un objet, et les opérations (addition, multiplication, etc.) sont modélisées par des classes.

Par exemple, l'expression mathématique suivante :

$$(2 + 3) + 4.5$$

est actuellement représentée par le code Java suivant :

```
new Addition(  
    new Addition(new Entier(2), new Entier(3)),  
    new Reel(4.5)  
);
```

Cette représentation correspond à la structure interne de l'expression (un arbre), mais elle présente plusieurs problèmes dans un contexte réel :

- le code est **verbeux** et difficile à écrire
- la lecture devient complexe lorsque les expressions sont imbriquées
- les utilisateurs doivent connaître en détail les classes internes (**Addition**, **Entier**, **Reel**)
- la construction d'expressions simples nécessite beaucoup de bruit syntaxique

Dans les projets utilisant cette bibliothèque, les ingénieurs doivent manipuler des expressions de plus en plus complexes, par exemple :

- génération automatique de formules
- transformations symboliques
- composition dynamique d'expressions

Dans ce contexte, l'API actuelle devient un frein : elle est **correcte d'un point de vue technique**, mais **peu ergonomique**.

L'équipe souhaite donc proposer une nouvelle manière de construire les expressions, plus naturelle et plus lisible, par exemple :

```
new Entier(2).add(3).add(4.5);
```

Cette évolution doit permettre de :

- simplifier l'écriture des expressions
- améliorer la lisibilité du code
- masquer les détails d'implémentation internes

Dans ce TP, vous devez concevoir cette API en garantissant : une bonne conception orientée objet, une API lisible et cohérente, et l'absence de duplication de code.

1 Modélisation des expressions

QUESTION. 1. Créer une classe abstraite :

```
abstract class Expression {  
    abstract double eval();  
}
```

Puis implémenter deux classes : **Entier** et **Reel** en respectant les contraintes suivantes :

- Respecter l'encapsulation
- Fournir un constructeur
- Implémenter la méthode `eval()`

2 Expressions composées

QUESTION. 2. Créer une classe **Addition** représentant la somme de deux expressions. L'évaluation doit être récursive, et les opérandes doivent être de type **Expression**.

```
class Addition extends Expression {  
    private Expression gauche;  
    private Expression droite;  
}
```

Exemple attendu :

```
Expression e = new Addition(new Entier(2), new Entier(3));  
System.out.println(e.eval()); // 5.0
```

3 Conception d'une API fluide avec surcharge

L'API actuelle est difficile à utiliser. Vous devez proposer une API permettant une écriture plus naturelle des expressions.

Ajouter dans la classe **Expression** les méthodes suivantes :

```
Expression add(Expression e)  
Expression add(int val)  
Expression add(double val)
```

et puis permettre l'écriture suivante :

```
Expression e = new Entier(2).add(3).add(4.5);  
System.out.println(e.eval());
```

Contraintes

- Éviter toute duplication de logique
- Concevoir une API cohérente
- Garantir la lisibilité du code

4 Extension : multiplication

Créer une classe `Multiplication`.

Ajouter les méthodes :

```
Expression mul(Expression e)
```

```
Expression mul(int val)
```

```
Expression mul(double val)
```

Exemple

```
Expression e = new Entier(2).add(3).mul(4);
```

```
System.out.println(e.eval());
```

5 Validation

Écrire un programme de test permettant de vérifier :

- des expressions simples
- des combinaisons d'opérations
- des enchaînements complexes